Abstract Windowing Toolkit
AWT

Introduction

- Java AWT (Abstract Windowing Toolkit) is an API to develop GUI or window-based application in java.
- Java AWT components are platform-dependent i.e. components are displayed according to the view of operating system.
- AWT is **heavyweight** i.e. its components uses the resources of system.
- The java.awt package provides classes for AWT API such as **TextField**, **Label**, **TextArea**, **RadioButton, CheckBox, Choice, List** etc.

Events

- An event is an object that describes a state change in a source.
- It can be generated as a consequence of a person interacting with the elements in a graphical user interface.
- Some of the activities that cause events to be generated are **pressing a button, entering a character via the keyboard, selecting an item in a list, and clicking the mouse**.
- Events may also occur that are not directly caused by interactions with a user interface.
- For example, an event may be generated when **a timer expires, a counter exceeds a value, a software or hardware failure occurs, or an operation is completed**.
- Events can be defined as needed and appropriate by application.

Event classes

- The Event classes that represent events are at the core of Java's event handling mechanism.
- Superclass of the Java event class hierarchy is **EventObject,** which is in **java.util.** for all events.

  Constructor is :

  EventObject(Object src)

  Here, src is the object that generates this event.
- EventObject contains two methods: getSource( ) and toString( ).

  1. The getSource( ) method returns the source of the event. General form is: Object.getSource( )
  2. The toString( ) returns the string equivalent of the event.

- EventObject is a super class of all events.
- AWTEvent is a super class of all AWT events that are handled by the delegation event model.
- The package java.awt.event defines several types of events that are generated by various user interface elements.

Event Classes in java.awt.event

- **ActionEvent**: Generated when a button is pressed, a list item is double clicked, or a menu item is selected.
- **AdjustmentEvent**: Generated when a scroll bar is manipulated.
- **ComponentEvent**: Generated when a component is hidden, moved, resized, or becomes visible.
- **ContainerEvent**: Generated when a component is added to or removed from a container.
- **FocusEvent**: Generated when a component gains or loses keyboard focus.

Event Classes in java.awt.event

- **InputEvent**: Abstract super class for all component input event classes.
- **ItemEvent**: Generated when a check box or list item is clicked; also occurs when a choice selection is made or a checkable menu item is selected or deselected.
- **KeyEvent:** Generated when input is received from the keyboard.
- **MouseEvent:** Generated when the mouse is dragged, moved, clicked, pressed, or released; also generated when the mouse enters or exits a component.
- **TextEvent:** Generated when the value of a text area or text field is changed.
- **WindowEvent:** Generated when a window is activated, closed, deactivated, de-iconified, iconified, opened, or quit.

Delegation event model

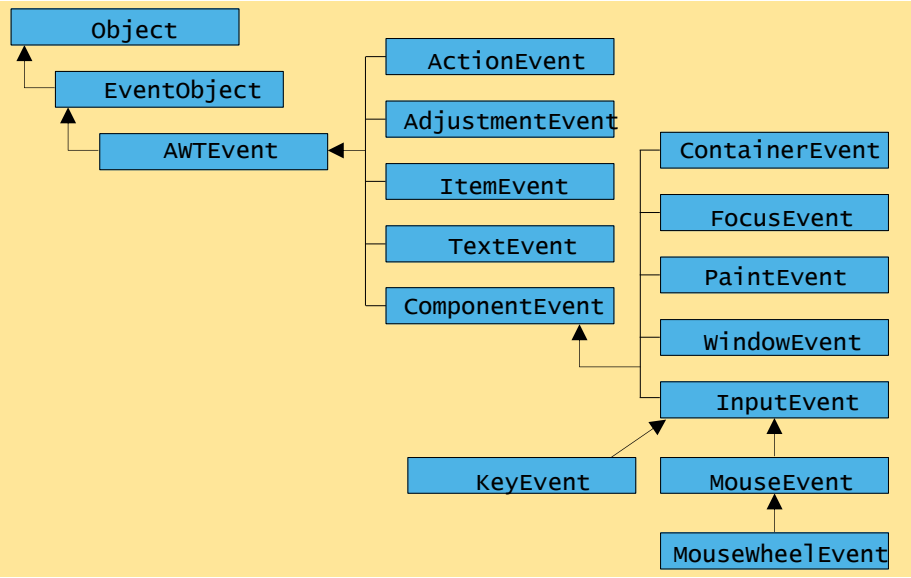
- Source generates an event & sends it to one or more listeners.
- Listener simply waits until it receives an event.
- Once received, listener processes the event & returns.
  - Advantage : application logic that processes event is clearly separated from the UI logic that generates those events.
  - An UI element is able to delegate the processing of an event to a separate piece of code (Event handler)

- This is a more efficient way to handle events than the design used by the old Java 1.0 approach. Previously, an event was propagated up the containment hierarchy until it was handled by a component.
- This required components to receive events that they did not process, and it wasted valuable time. The delegation event model eliminates this overhead.

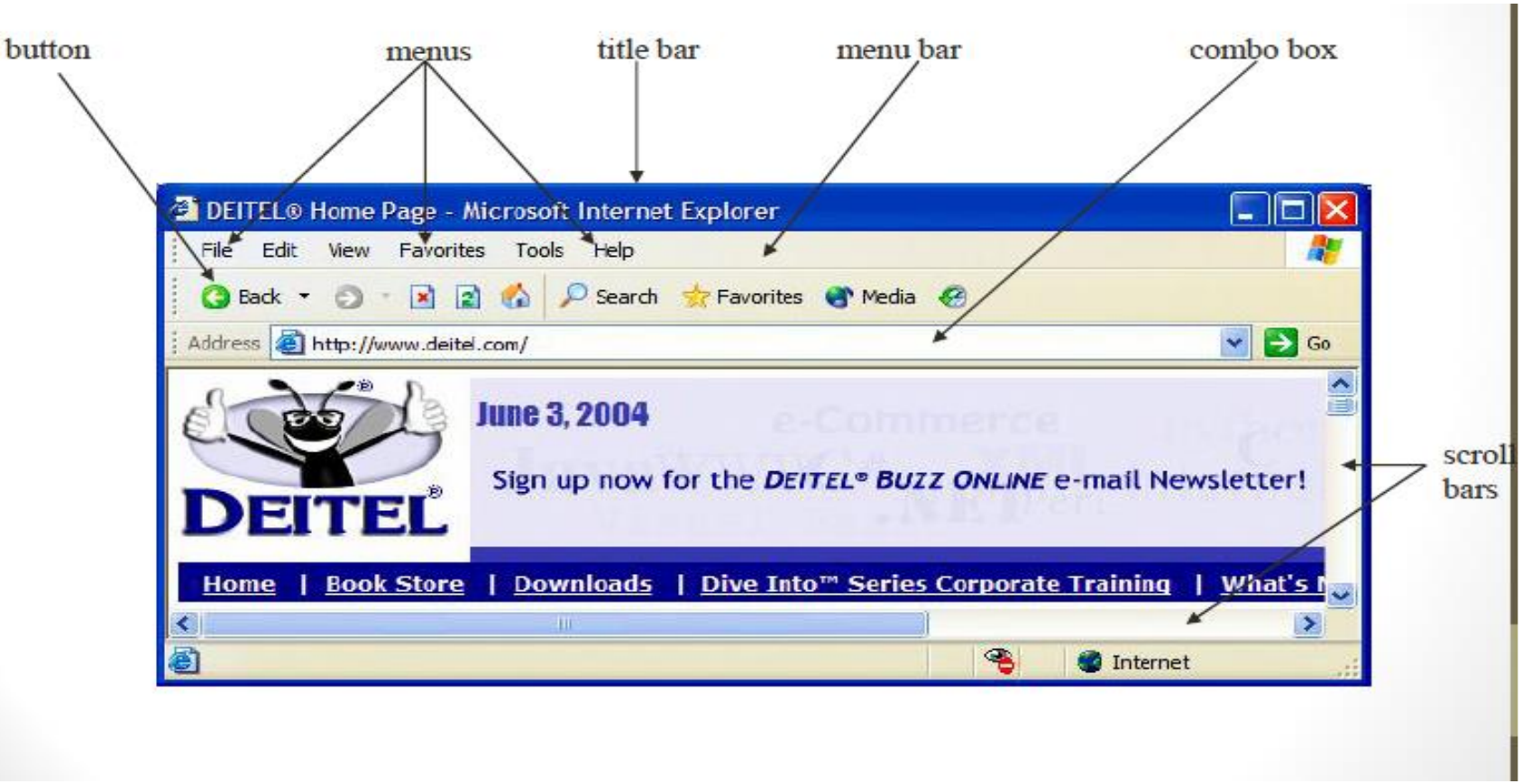
Note

- Java also allows you to process events without using the delegation event model.
- This can be done by extending an AWT component.

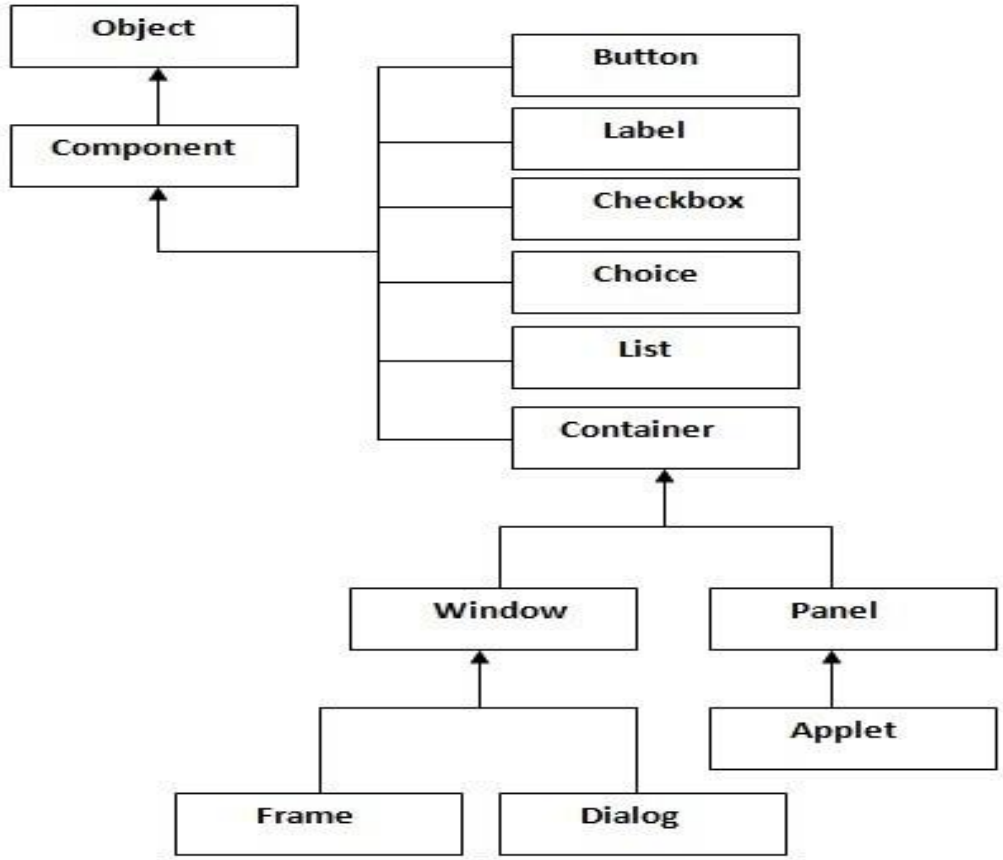
Some event classes of package `java.awt.event`



GUI


button
menus
title bar
menu bar
combo box
DEITEL® Home Page - Microsoft Internet Explorer
File Edit View Favorites Tools Help
Back Search Favorites Media
Address http://www.deitel.com/
Go
DEITEL®
June 3, 2004
Sign up now for the DEITEL® BUZZ ONLINE e-mail Newsletter!
Home | Book Store | Downloads | Dive Into™ Series Corporate Training |
Internet
scroll bars

AWT Hierarchy



Object

- The Object class is the top most class and parent of all the classes in java by default.
- Every class in java is directly or indirectly derived from the object class.

Component

- The Component is abstract class that encapsulates all the attributes of visual component.
- All User interface (UI) elements that are displayed on screen are subclasses of Component.

Component is responsible for remembering the current foreground and background color and the currently selected text font.

Methods of Component class

Method	Description
public void add(Component c)	inserts a component on this component.
public void setSize(int width, int height)	sets the size (width and height) of the component.
public void setLayout(LayoutManager m)	defines the layout manager for the component.
public void setVisible(boolean status)	changes the visibility of the component, by default false.
void remove(Component c)	Remove a component
void setBounds(int x,int y, int width, int height)	Set the location and size of single component and useful only with null layout.

Container

- The Container is a component in AWT that can contain another components like buttons, textfields, labels etc.
- The classes that extends Container class are known as container such as Frame, Dialog and Panel.
- Container is **responsible for laying out any components** that it contains through the use of layout managers.
- **Methods:**
  - void setFont(Font f)
  - void setLayout(LayoutManager mgr)

Panel

- Panel class is concrete class it doesn’t add new methods.
- The Panel is the container that **doesn't contain title bar and menu bars and Borders**.
- It can have other components like button, textfield etc.

An Applet is a Panel which is a Container

java.lang.Object
| +----java.awt.Component
| +----java.awt.Container
| +----java.awt.Panel
| +----java.applet.Applet

Window

- The window is the container that have **no borders and menu bars.**
- You must use frame, dialog or another window for creating a window.

Frame

- It is subclass of **Window.**
- The Frame is the container that contain title bar and can have menu bars,borders, and resizing corners.
- It can have other components like button, textfield, etc.
- Methods:
  - void setTitle(String title)
  - void setBackground(Color bgcolor)

Working with Frame Window

- Extends Frame class
- Constructor are:
  - Frame()
  - Frame(String title)
- Setting and Getting window size:
  - void setSize(int width, int height)
  - void setSize(Dimension newsize)
- Showing and Hiding Frame
  - void setVisible(boolean visibleFlag)

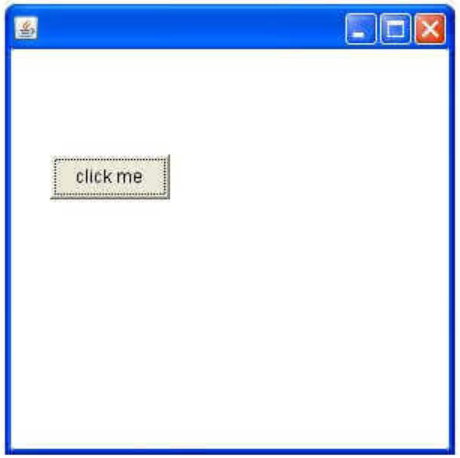
Frame Class

- We can create stand-alone AWT based applications.
- A Frame provides main window for the GUI application.
- There are two ways to create a Frame :
  1. By instantiating Frame Class
  2. By extending Frame class

Program using Frames

```
import java.awt.*;
class FirstFrame{
FirstFrame(){
Frame f=new Frame();
Button b=new Button("click me");
b.setBounds(30,50,80,30);
f.add(b);
f.setSize(300,300);
f.setLayout(null);
f.setVisible(true);  }
public static void main(String args[]){

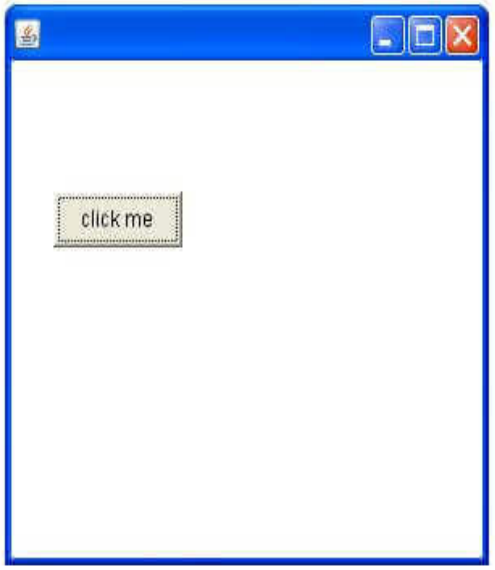
FirstFrame f=new FirstFrame();
}
}
```



Program using Frames

```
import java.awt.*;
class First extends Frame{
First(){
Button b=new Button("click me");
b.setBounds(30,100,80,30);
add(b);
setSize(300,300);
setLayout(null);
setVisible(true);
}
```

```
public static void main
(String args[]){
First f=new First();
}
}
```



The AWT class hierarchy

The AWT classes are contained in the java.awt package. It is one of Java's largest packages. Some of the AWT classes.

AWT Classes

1. **AWTEvent**: Encapsulates AWT events.
2. **AWTEventMulticaster**: Dispatches events to multiple listeners.
3. **Border Layout**: The border layout manager. Border layouts use five components: North, South, East, West, and Center.
4. **Button**: Creates a push button control.
5. **Canvas:** A blank, semantics-free window.
6. **Card Layout**: The card layout manager. Card layouts emulate index cards. Only the one on top is showing.

7. **Checkbox**: Creates a check box control.
8. **CheckboxGroup:** Creates a group of check box controls.
9. **CheckboxMenuItem:** Creates a non/off menu item.
10. **Choice:** Creates a pop-up list.
11. **Color:** Manages colors in a portable, platform-independent fashion.
12. **Component:** An abstract superclass for various AWT components.
13. **Container:** A subclass of Component that can hold other components.
14. **Cursor**: Encapsulates a bit mapped cursor.
15. **Dialog:** Creates a dialog window.
16. **Dimension:** Specifies the dimensions of an object. The width is stored in width, and the height is stored in height.
17. **Event:** Encapsulates events.
18. **EventQueue**: Queues events.
19. **FileDialog**: Creates a window from which a file can be selected.
20. **FlowLayout**: Theflow layout manager. Flow layout positions components left to right, top to bottom.

21. **Font**: Encapsulates a type font.
22. **FontMetrics**: Encapsulates various information related to a font. This information helps you display text in a window.
23. **Frame**: Creates a standard window that has a title bar, resize corners, and a menu bar.
24. **Graphics**: Encapsulates the graphics context. This context is used by various output methods to display output in a window.
25. **Graphics Device**: Describes a graphics device such as a screen or printer.
26. **Graphics Environment**: Describes the collection of available Font and Graphics Device objects.
27. **GridBag Constraints**: Defines various constraints relating to the GridBagLayout class.
28. **GridBag Layout**: The grid bag layout manager. Grid bag layout displays components subject to the constraints specified by GridBag Constraints.
29. **Grid Layout**: Thegrid layout manager. Grid layout displays components in a two-dimensional grid.

30. **Scrollbar:** Creates a scroll bar control.
31. **ScrollPane:** A container that provides horizontaland/or vertical scrollbars for another component.
32. **SystemColor:** Contains the colors of GUI widgets such as windows, scrollbars, text, and others.
33. **TextArea:** Creates a multi line edit control.
34. **TextComponent:** A superclass for TextArea and TextField.
35. **TextField:** Creates a single-line edit control.
36. **Toolkit:** Abstract class implemented by the AWT.
37. **Window:** Creates a window with no frame, no menubar, and no title.

Controls

- **Labels**
- **Buttons**
- **Checkbox**
- **CheckboxGroup**
- **Textfield**
- **TextFieldArea**
- **ScollBar**

Label

- The easiest control to use is a label.
- A label is an object of type Label, and it contains a string, which it displays.
- Labels are passive controls that do not support any interaction with the user.

Labels

- **Label defines the following constructors:**
- **Label( )**
- **Label(String str)**
- **Label(String str, int how)**
- The first version creates a blank label.
- The second version creates a label that contains the string specified by *str*. This string is left-justified.
- The third version creates a label that contains the string specified by *str* using the alignment specified by *how*. The value of how must be one of these three constants: Label.LEFT, Label.RIGHT, or Label.CENTER.

Label

- Methods
- void setText(String str)
- String getText( )
- void setAlignment(int how)
- int getAlignment( )

Controls

```
// Demonstrate Labels
import java.awt.*;

public class LabelDemo
extends Frame
{
public LabelDemo()
{
Label one = new Label("One");
Label two = new Label("Two");
Label three = new Label("Three");
// add labels to applet window
add(one);
add(two);
add(three);
}
}
```

Buttons

- The most widely used control is the push button.
- A push button is a component that contains a label and that generates an event when it is pressed.
- Push buttons are objects of type Button.
- Button defines these two constructors:
- **Button( )**
- **Button(String str)**

Buttons

- **String getLabel()**
- **void setLabel(String str)**
- **void setEnabled(Boolean enable)**
- **Void addActionListener(ActionListener l)**
- **void removeActionListener(ActionListener l)**
- **String getActionCommand()**
- **void setActionCommand(String Cmd)**

```
// Demonstrate Buttons
import java.awt.*;

public class ButtonDemo extends
Frame
{
String msg = "";
Button yes, no, maybe;
public ButtonDemo()
{
yes = new Button("Yes");
no = new Button("No");
maybe = new Button(“Understand");
add(yes);
add(no);
add(maybe);
}
public void paint(Graphics g)
{
g.drawString(msg, 6, 100);
}
}
```

Check Boxes

- A check box is a control that is used to turn an option on or off.
- It consists of a small box that can either contain a check mark or not.
- There is a label associated with each check box that describes what option the box represents.
- We change the state of a check box by clicking on it. Check boxes can be used individually or as part of a group.

Checkbox constructors:

- Checkbox( )
- Checkbox(String str)
- Checkbox(String str, boolean on)
- Checkbox(String str, boolean on, CheckboxGroup cbGroup)
- Checkbox(String str, CheckboxGroup cbGroup, boolean on)

Methods

- boolean getState( )
- void setState(boolean on)
- String getLabel( )
- void setLabel(String str)
- void addItemListener(ItemListener l)
- void removeItemListener(ItemListener l)

```
// Demonstrate check boxes.
import java.awt.*;

public class CheckboxDemo extends
Frame
{
String msg = "";
Checkbox Win98, winNT, solaris, mac;
public CheckboxDemo()
{
Win98 = new Checkbox("Windows 98/XP",
null, true);
winNT = new Checkbox("Windows
NT/2000");
solaris = new Checkbox("Solaris");
mac = new Checkbox("MacOS");
add(Win98);
add(winNT);
add(solaris);
add(mac);
}
```


Applet Viewer: Checkb...
Applet
Windows 98/XP
Windows NT/2000
Solaris
MacOS
Applet started.

Checkbox Group

- It is possible to create a set of mutually exclusive check boxes in which one and only one check box in the group can be checked at any one time.
- These check boxes are often called radio button.
- Check box groups are objects of type **CheckboxGroup**.
- Only the default constructor is defined, which creates an empty group.

Methods

Checkbox getSelectedCheckbox( )

void setSelectedCheckbox(Checkbox wh)

```
import java.awt.*;

public class CBGroup extends Frame
{
String msg = "";
Checkbox Win98, winNT,
solaris, mac;
CheckboxGroup cbg;
public CBGroup()
{
cbg = new CheckboxGroup();
Win98 = new Checkbox("Windows 98/XP", cbg,
true);
winNT = new Checkbox("Windows NT/2000", cbg,
false);
solaris = new Checkbox("Solaris", cbg,
false);
mac = new Checkbox("MacOS", cbg, false);
add(Win98); add(winNT);
add(solaris); add(mac);
}
```


Applet Viewer: CBGroup
Applet
Windows 98/XP
Windows NT/2000
Solaris
MacOS
Current selection: Windows 98/XP
Applet started.

Choice Controls

- The Choice class is used to create a pop-up list of items from which the user may choose.
- Thus, a Choice control is a form of menu.
- Each item in the list is a string that appears as a left justified label in the order it is added to the Choice object.

Methods

void add(String name)

String getSelectedItem( )

int getSelectedIndex( )

int getItemCount( )

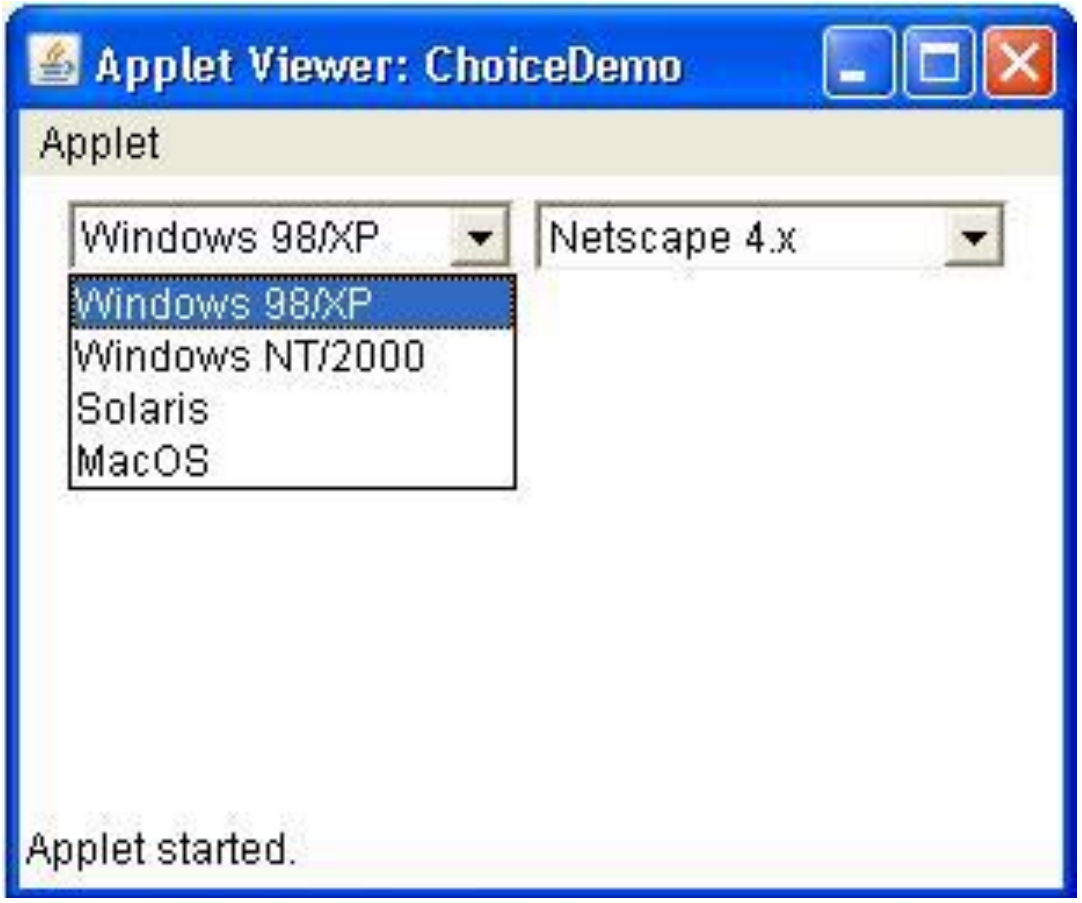
void select(int index)

void select(String name)

String getItem(int index)

```
import java.awt.*;

public class ChoiceDemo extends
Frame
{
Choice os, browser;
String msg = "";
public ChoiceDemo()
{
os = new Choice();
browser = new Choice();
os.add("Windows 98/XP");
os.add("Windows NT/2000");
os.add("Solaris");
os.add("MacOS");
browser.add("Netscape 3.x");
browser.add("Netscape 4.x");
browser.add("Netscape 5.x");
browser.add("Netscape 6.x");
browser.add("Internet Explorer 4.0");
browser.add("Internet Explorer 5.0");
browser.add("Internet Explorer 6.0");
browser.add("Lynx 2.4");
browser.select("Netscape 4.x");
add(os);
add(browser);
}
```


Applet Viewer: ChoiceDemo
Applet
Windows 98/XP
Netscape 4.x
Windows 98/XP
Windows NT/2000
Solaris
MacOS
Applet started.

Lists

- The List class provides a compact, multiple-choice, scrolling selection list.
- Unlike the Choice object, which shows only the single selected item in the menu, a List object can be constructed to show any number of choices in the visible Window.
- It can also be created to allow multiple selections.

List

- **List( )**
- **List(int numRows)**
- **List(int numRows, boolean multipleSelect)**

- The first version creates a List control that allows only one item to be selected at any one time.
- In the second form, the value of *numRows* specifies the number of entries in the list that will always be visible (others can be scrolled into view as needed).
- In the third form, if *multipleSelect* is true, then the user may select two or more items at a time.

Methods

void add(String name)

void add(String name, int index)

String getSelectedItem( )

int getSelectedIndex( )

String[ ] getSelectedItems( )

int[ ] getSelectedIndexes( )

int getItemCount( )

void select(int index)

String getItem(int index)


Applet Viewer: ListDemo
Applet
Windows 98/XP
Windows NT/2000
Solaris
MacOS
Netscape 4.x
Netscape 5.x
Netscape 6.x
Current OS:
Current Browser: Netscape 4.x
Applet started.

ScrollBars

- Scroll bars are used to select continuous values between a specified minimum and maximum.
- Scroll bars may be **oriented horizontally or vertically.**
- A scroll bar is actually a composite of several individual parts.
  - slider box (or thumb) for the scroll bar.
- The slider box can be dragged by the user to a new position, this action translates into some form of **page up and page down.**

Constructors

- **Scrollbar( )**
- **Scrollbar(int style)**
- **Scrollbar(int style, int iValue, int tSize, int min, int max)**

- The first form creates a **vertical scroll bar**.
- The second and third forms allow us to specify *style* **Scrollbar.VERTICAL, Scrollbar.HORIZONTAL**.
- In the third form, the **initial value** of the scroll bar is passed in ***iValue*.** The number of units represented by the **height of the thumb** is passed in ***tSize***. The minimum and maximum values for the scroll bar are specified by min and max.

Methods

void setValues(int iValue, int tSize, int min, int max)

int getValue( )

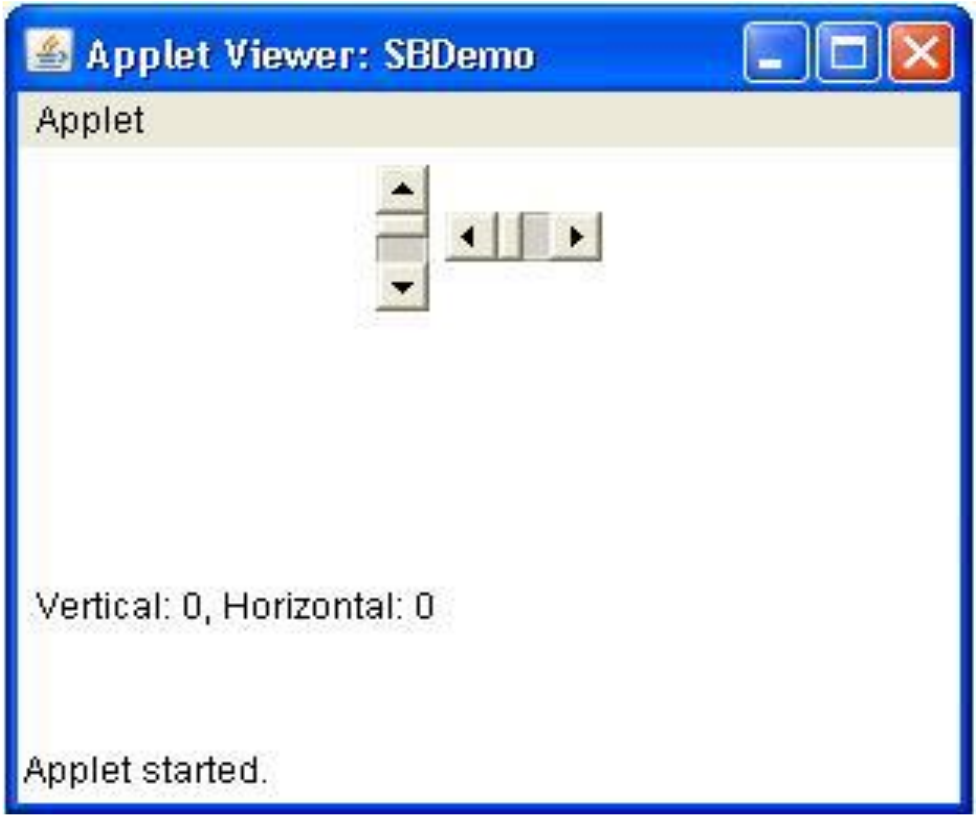
void setValue(int newValue)

int getMinimum( )

int getMaximum( )

void setUnitIncrement(int newIncr)

void setBlockIncrement(int newIncr)


Applet Viewer: SBDemo
Applet
Vertical: 0, Horizontal: 0
Applet started.

TextField

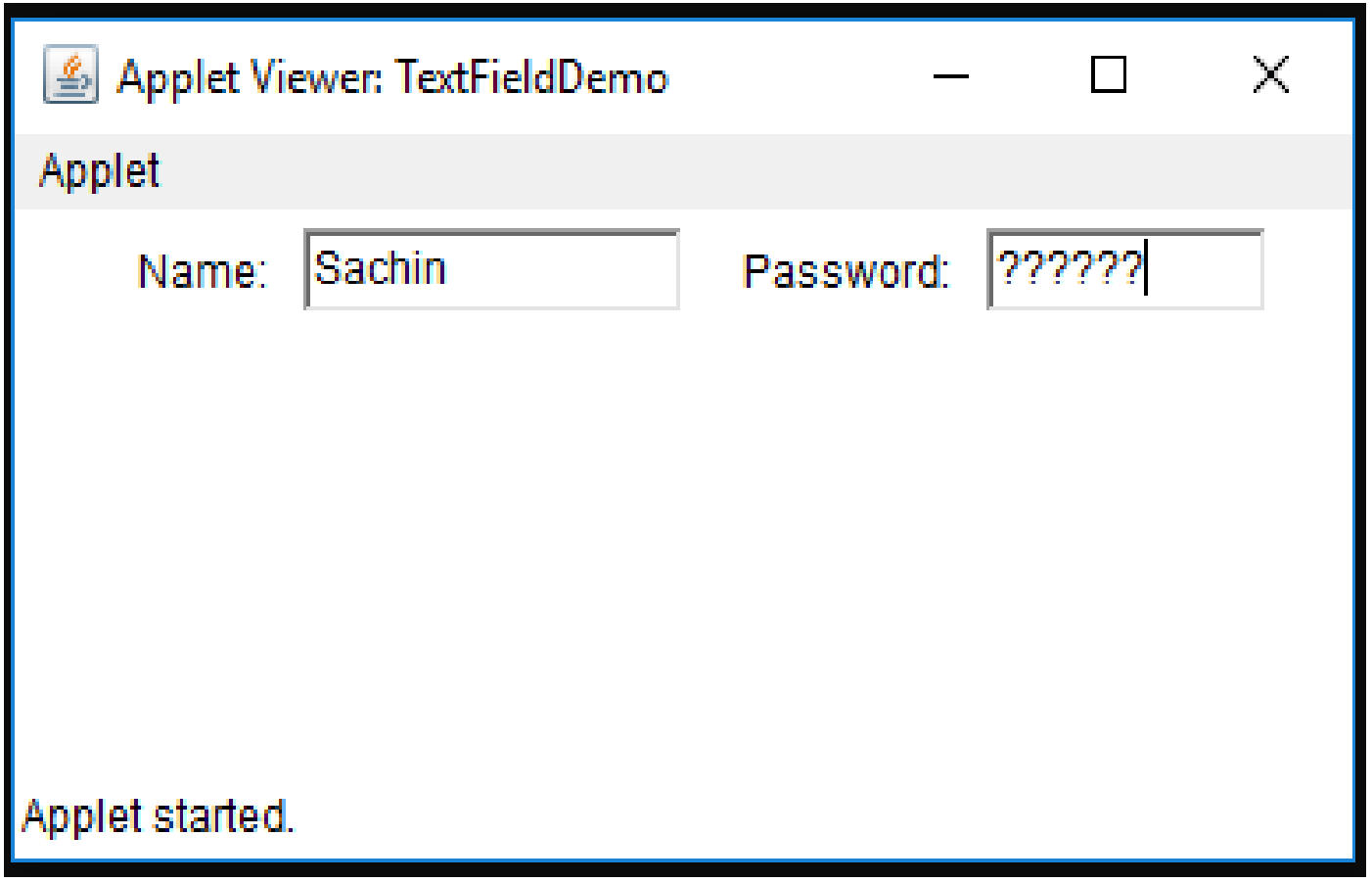
- The TextField class implements a single-line text-entry area, called an **edit control**.
- Text fields allow the user to enter strings and to edit the text using the arrow keys, cut and paste keys, and mouse selections.
- TextField is a subclass of **TextComponent.**

TextField Constructors

- TextField( )
- TextField(int numChars)
- TextField(String str)
- TextField(String str, int numChars)

TextField Methods

- String getText( )
- void setText(String str)
- String getSelectedText( )
- void select(int startIndex, int endIndex)
- boolean isEditable( )
- void setEditable(boolean canEdit)
- void setEchoChar(char ch)
- boolean echoCharIsSet( )
- char getEchoChar( )


Applet Viewer: TextFieldDemo
Applet
Name:
Sachin
Password:
??????
Applet started.

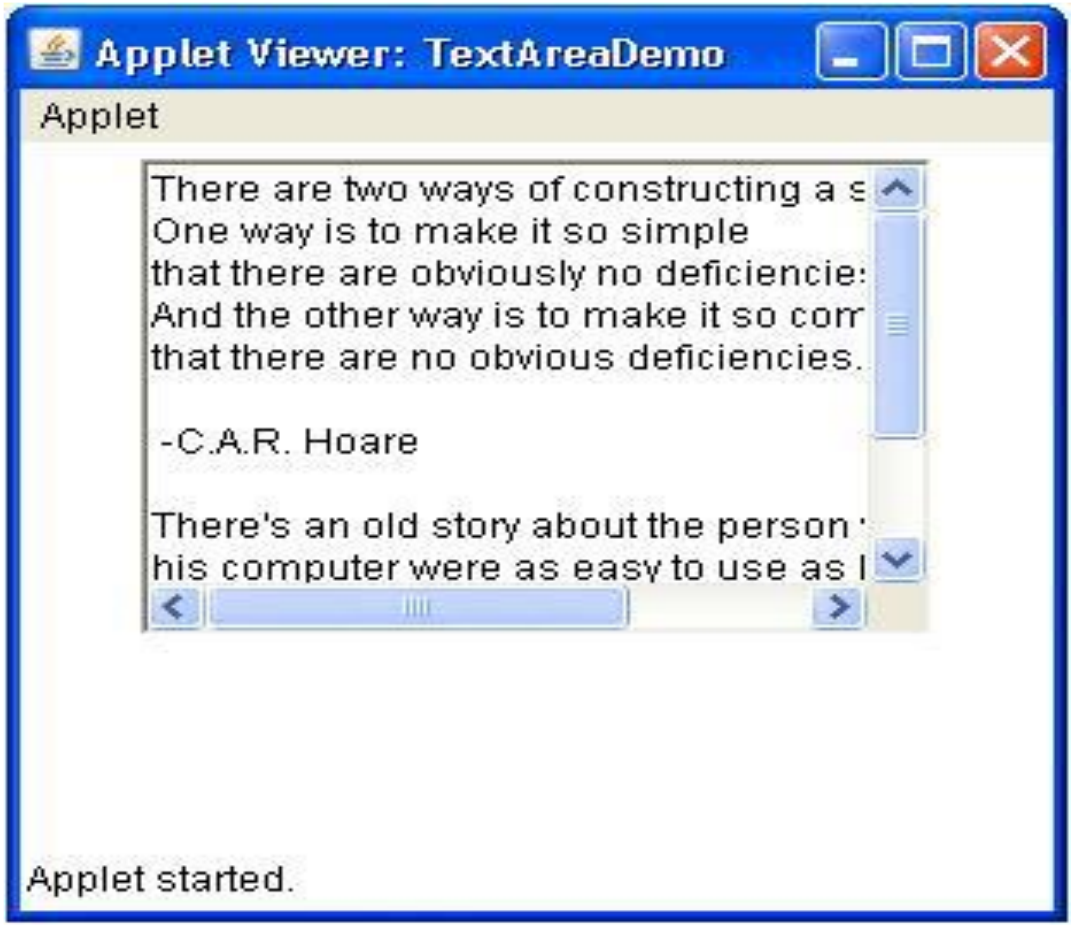
TextArea

- Sometimes a single line of text input is not enough for a given task. To handle these situations, the AWT includes a simple multiline editor called TextArea.
- Following are the constructors for TextArea:
  - TextArea( )
  - TextArea(int numLines, int numChars)
  - TextArea(String str)
  - TextArea(String str, int numLines, int numChars)
  - TextArea(String str, int numLines, int numChars, int sBars)

sBars must be one of these values: **SCROLLBARS_BOTH, SCROLLBARS_NONE,SCROLLBARS_HORIZONTAL_ONLY, SCROLLBARS_VERTICAL_ONLY**

Methods

- TextArea is a subclass of TextComponent.
- Therefore, it supports the getText( ), setText( ), getSelectedText( ), select( ), isEditable( ), and setEditable( ) methods as of TextField.
- TextArea adds the following methods:
- **void append(String str)**
- **void insert(String str, int index)**
- **void replaceRange(String str, int startIndex, int endIndex)**


Applet Viewer: TextAreaDemo
Applet
There are two ways of constructing a s
One way is to make it so simple
that there are obviously no deficiencie
And the other way is to make it so com
that there are no obvious deficiencies.
-C.A.R. Hoare
There's an old story about the person
his computer were as easy to use as
Applet started.

Layout Managers

- Layout means arranging the components within the container.
- The task of lay outing can be done automatically by the Layout manager.
- The layout manager is set by the setLayout( ) method.
- If no call to setLayout( ) is made, then the default layout manager is used.
- Whenever a container is resized (or sized for the first time), the layout manager is used to position each of the components within it.

- The setLayout( ) method has the following general form:
- void setLayout(LayoutManager layoutObj)
- Here, *layoutObj* is a reference to the desired layout manager.
- If we wish to disable the layout manager and position components manually, pass null for *layoutObj*.

LayoutManager

- **LayoutManager** is an interface that is implemented by all the classes of layout managers. There are following classes that represents the layout managers:
- FlowLayout
- BorderLayout
- GridLayout
- CardLayout
- GridBagLayout

FlowLayout

- FlowLayout is the default layout manager.
- FlowLayout implements a simple layout style, which is similar to how words flow in a text editor.
- Components are laid out from the upper-left corner, left to right and top to bottom.
- When no more components fit on a line, the next one appears on the next line.
- A small space is left between each component, above and below, as well as left and right.

FlowLayout Constructors

- **FlowLayout( )**
- **FlowLayout(int how)**
- **FlowLayout(int how, int horz, int vert)**
- The first is default, which centers components and leaves **five pixels** of space between each component**.**
- The second form lets us specify how each line is aligned. Valid values for how are as follows:
  - FlowLayout.LEFT
  - FlowLayout.CENTER
  - FlowLayout.RIGHT
- The third form allows us to specify the horizontal and vertical space left between components

FlowLayout Methods

- int getAlignment()
- int getHgap()
- int getVgap()
- int setAlignment(int align)
- int setHgap(int hgap)
- int setVgap(int vgap)

```
public class FlowLayoutDemo extends Applet
{
Checkbox Win98, winNT, solaris, mac;
public void init()
{
Win98 = new Checkbox("Windows 98/XP", null, true);
winNT = new Checkbox("Windows NT/2000");
solaris = new Checkbox("Solaris");
mac = new Checkbox("MacOS");
setLayout(new FlowLayout(FlowLayout.CENTER));
add(Win98); add(winNT);add(solaris);add(mac);
}}
```


Applet Viewer: FlowLa...
Applet
Windows 98/XP
Windows NT/2000
Solaris
MacOS
Applet started.

BorderLayout

- The BorderLayout class implements a common layout style for top-level windows.
- It has four narrow, fixed-width components at the edges and one large area in the center.
- The four sides are referred to as
  - north,
  - south,
  - east, and
  - west.
  - The middle area is called the center.

BorderLayout Constructors

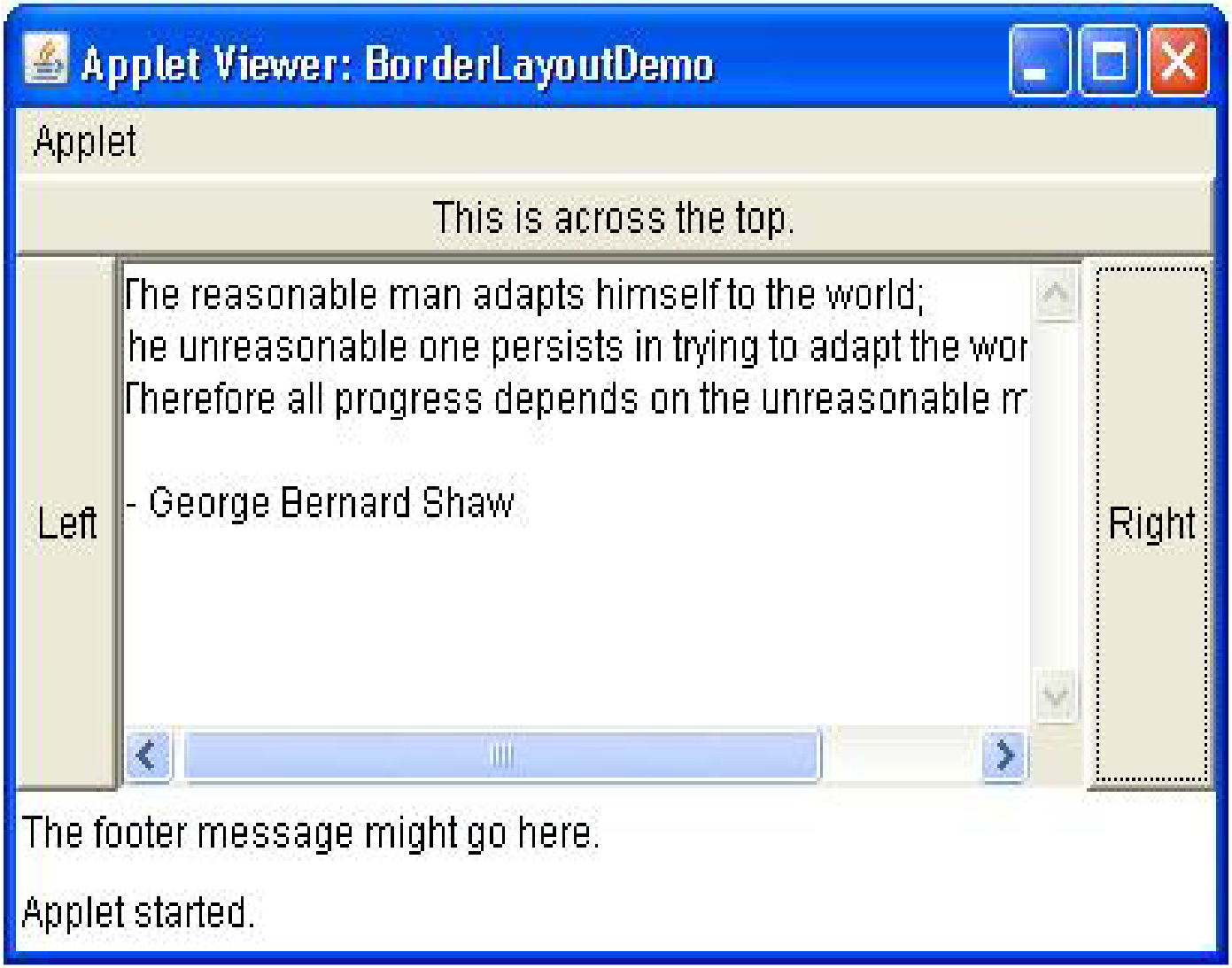
- BorderLayout( )
- BorderLayout(int horz, int vert)

- The first form creates a default border layout.
- The second allows us to specify the horizontal and vertical space left between components in *horz* and *vert*, respectively.

BorderLayout

- BorderLayout defines the following constants that specify the regions:
- **BorderLayout.CENTER**
- **BorderLayout.SOUTH**
- **BorderLayout.EAST**
- **BorderLayout.WEST**
- **BorderLayout.NORTH**
- To add components, we use these constants with the following form of add( ), which is defined by Container:
- **void add(Component compObj, Object region);**
- Here, *compObj* is the component to be added, and region specifies where the component will be added.

```
public class BorderLayoutDemo extends Applet {
public void init() {
setLayout(new BorderLayout());
add(new Button("This is across the top."), BorderLayout.NORTH);
add(new Label("The footer message."), BorderLayout.SOUTH);
add(new Button("Right"), BorderLayout.EAST);
add(new Button("Left"), BorderLayout.WEST);
String msg = "The reasonable man adapts himself to the world;\n" +
"the unreasonable one persists in trying to adapt the world to himself.\n" +
"Therefore all progress depends on the unreasonable man.\n\n" + " - George Bernard Shaw\n\n";
add(new TextArea(msg), BorderLayout.CENTER);
}
}
```


Applet Viewer: BorderLayoutDemo
Applet
This is across the top.
Left
The reasonable man adapts himself to the world;
he unreasonable one persists in trying to adapt the wor
Therefore all progress depends on the unreasonable m
- George Bernard Shaw
Right
The footer message might go here.
Applet started.

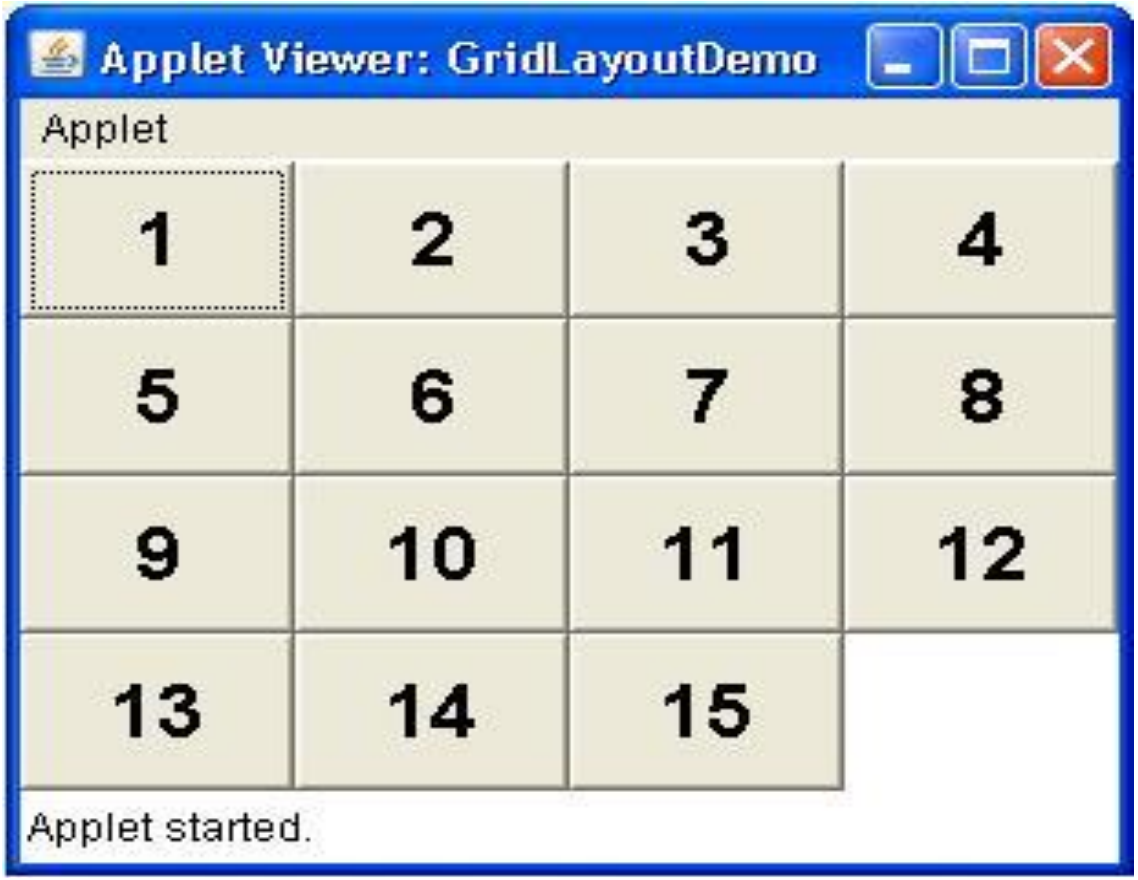
GridLayout

- GridLayout lays out components in a two-dimensional grid.
- When we instantiate a GridLayout, we define the number of rows and columns.

GridLayout constructors

- GridLayout( )
- GridLayout(int numRows, int numColumns)
- GridLayout(int numRows, int numColumns, int horz, int vert)
- The first form creates a single-column grid layout.
- The second creates a grid layout with specified number of rows and columns.
- **Either numRows or numColumns can be zero**.
- Specifying *numRows* as zero allows for unlimited-length columns.
- Specifying *numColumns* as zero allows for unlimited-length rows.

```
public class GridLayoutDemo extends Applet {
static final int n = 4;
public void init(){
setLayout(new GridLayout(n, n));
setFont(new Font("SansSerif", Font.BOLD, 24));
for(int i = 0; i < n; i++){
for(int j = 0; j < n; j++){
int k = i * n + j;
if(k > 0)
add(new Button("" + k));
} }}}
```


Applet Viewer: GridLayoutDemo
Applet
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
Applet started.

CardLayout

- The CardLayout class is unique among the other layout managers in that it stores several different layouts.
- Each layout can be thought of as being on a separate index card in a deck that can be shuffled so that any card is on top at a given time.
- This can be useful for user interfaces with optional components that can be dynamically enabled and disabled upon user input.
- We can prepare the other layouts and have them hidden, ready to be activated when needed.

- CardLayout provides these two constructors:
- **CardLayout( )**
- **CardLayout(int horz, int vert)**
- The first form creates a default card layout.
- The second form allows us to specify the horizontal and vertical space left between components.

Methods

- void add(Component panelObj, Object name);
- Here **name** is a string that specifies the name of the card whose panel is specified by *panelObj*. After we have created a deck, our program activates a card by calling one of the following methods:
  - void first(Container deck)
  - void last(Container deck)
  - void next(Container deck)
  - void previous(Container deck)
  - void show(Container deck, String cardName)
- *deck* is a reference to the container (usually a panel) that holds the cards, and *cardName* is the name of a card.

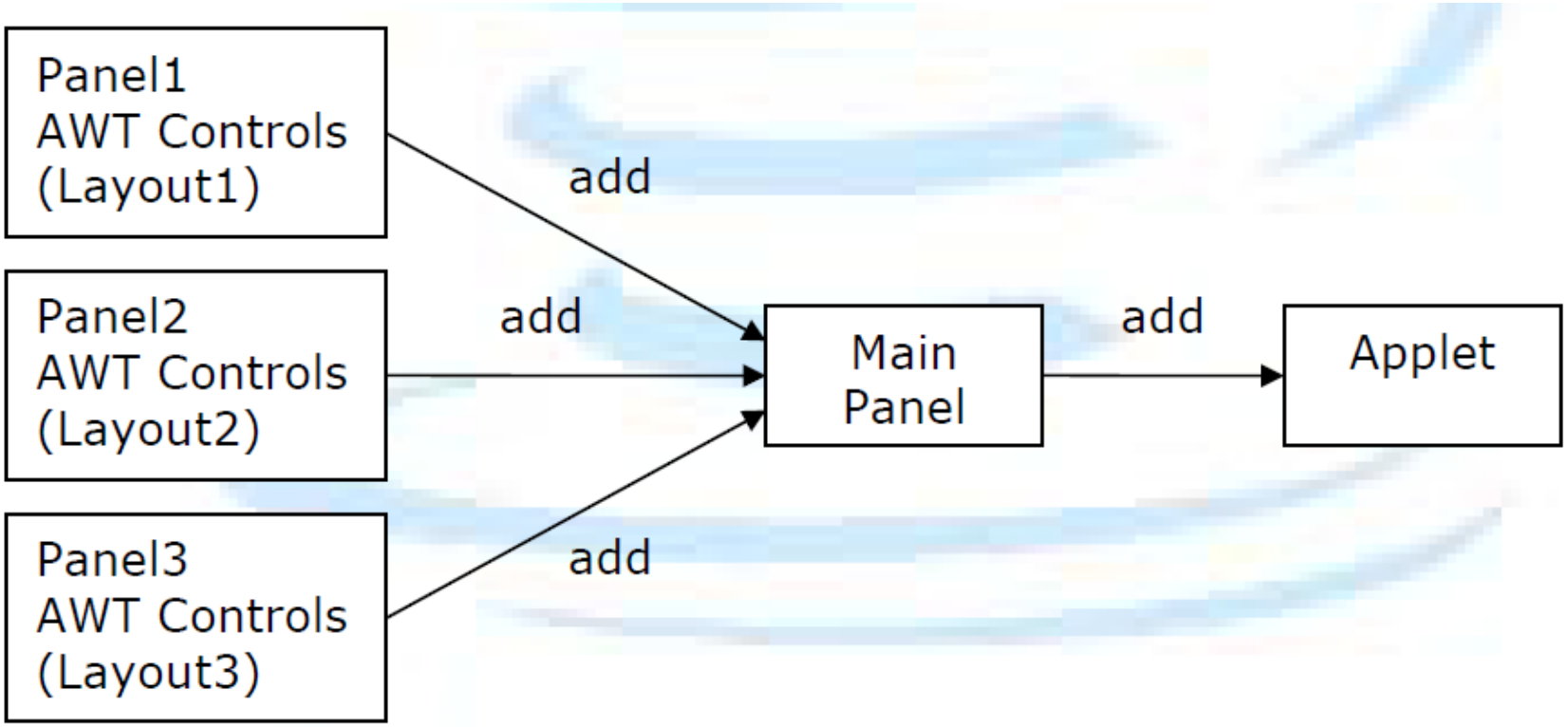


Fig. Creation of card layout

- // Demonstrate CardLayout.

```
import java.awt.*;
import java.awt.event.*;
public class CardLayoutExample extends Frame implements
 ActionListener{
CardLayout card;
Button b1,b2,b3;
    CardLayoutExample(){
     card=new CardLayout(40,30);
      setLayout(card);
```

```
b1=new Button("Apple");
    b2=new Button("Boy");
    b3=new Button("Cat");
    b1.addActionListener(this);
    b2.addActionListener(this);
    b3.addActionListener(this);
    add(b1,"card1"); add(b2,"card2"); add(b3,"card3");     }
  public void actionPerformed(ActionEvent e) {
  card.next(this);
  }
  public static void main(String[] args) {
    CardLayoutExample cl=new CardLayoutExample();
    cl.setSize(400,400);
    cl.setVisible(true);
       }  }
```

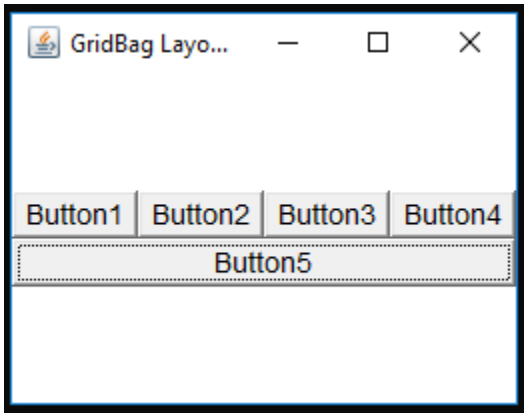

Apple

GridBagLayout

- Each GridBagLayout object maintains a dynamic rectangular grid of cells, with each component occupying one or more cells, called its *display area*.
- Each component managed by a grid bag layout is associated with an instance of GridBagConstraints that specifies how the component is laid out within its display area.

- For customize a GridBagConstraints object by setting one or more of its instance variables:
- gridx, gridy: Specifies the cell at the upper left of the component's display area, where the upper-left-most cell has address gridx = 0, gridy = 0.
- gridwidth, gridheight: Specifies the number of cells in a row (for gridwidth) or column (for gridheight) in the component's display area. The default value is 1.
- fill: Used when the component's display area is larger than the component's requested size to determine whether (and how) to resize the component.

```
import java.awt.*;
import java.util.*;
import java.applet.Applet;
public class GridBagEx1 extends Applet {
   protected void makebutton(String name,
                             GridBagLayout gridbag,
                             GridBagConstraints c) {
      Button button = new Button(name);
      gridbag.setConstraints(button, c);
      add(button);
   }
   public void init() {
      GridBagLayout gridbag = new GridBagLayout();
      GridBagConstraints c = new GridBagConstraints();
      setLayout(gridbag);
```




```
c.fill = GridBagConstraints.BOTH;
        c.weightx = 1.0;
        makebutton("Button1", gridbag, c);
        makebutton("Button2", gridbag, c);
        makebutton("Button3", gridbag, c);
        c.gridwidth = GridBagConstraints.REMAINDER; //end row
        makebutton("Button4", gridbag, c);
        c.weightx = 0.0;                    //reset to the default
        makebutton("Button5", gridbag, c); //another row
}
        public static void main(String args[]) {
            Frame f = new Frame("GridBag Layout Example");
            GridBagEx1 ex1 = new GridBagEx1();
            ex1.init();          f.add("Center", ex1);       f.pack();
            f.resize(f.preferredSize());
            f.show();    } }
```

Menu Bars and Menus

- A menu bar displays a list of top-level menu choices. Each choice is associated with a dropdown menu.
- This concept is implemented in Java by the following classes:
  - **MenuBar**, **Menu**, and **MenuItem**.
- In general, a menu bar contains one or more Menu objects. Each Menu object contains a list of MenuItem objects. Each MenuItem object represents something that can be selected by the user.

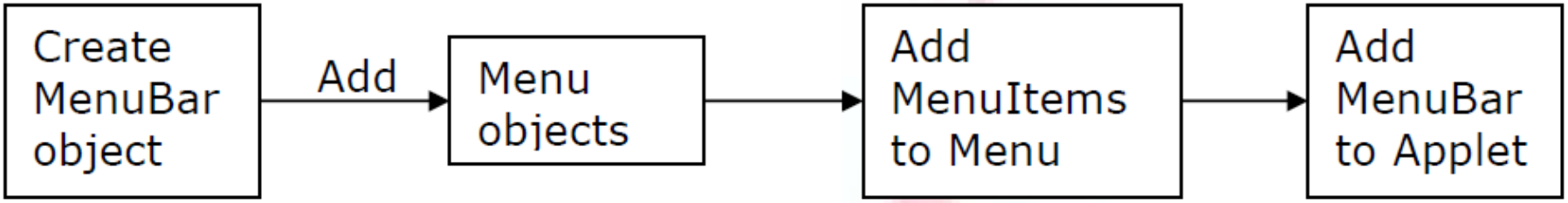


Fig. Creating a menu on Frame

- MenuBar Class Defines only default constructor.
- Menu Class Constructors
  - Menu( )
  - Menu(String optionName)
  - Menu(String optionName, boolean removable)
- Here, *optionName* specifies the name of the menu selection.
- Individual menu items constructors:
  - MenuItem( )
  - MenuItem(String *itemName*)
  - MenuItem(String *itemName*, MenuShortcut *keyAccel*)

Methods

- Disable or enable a menu item by using:
  - void setEnabled(boolean *enabledFlag*)
  - boolean isEnabled( )
- Label set and get using:
  - void setLabel(String *newName*)
  - String getLabel( )
- Checkable menu item by using a subclass of **MenuItem** called **CheckboxMenuItem**. :
  - CheckboxMenuItem( )
  - CheckboxMenuItem(String *itemName*)
  - CheckboxMenuItem(String *itemName*, boolean *on*)

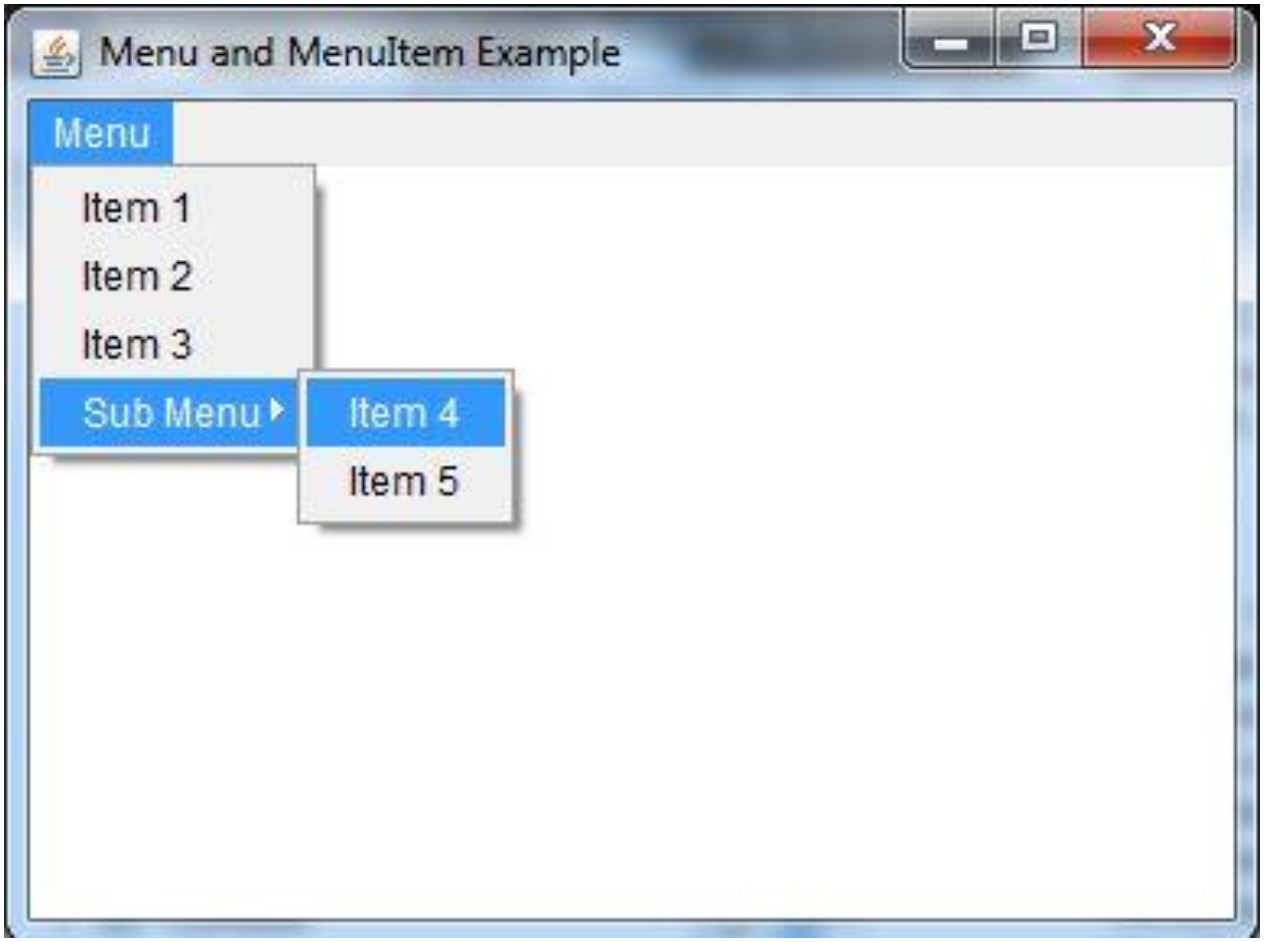
Methods

- Status about checkable MenuItem:
  - boolean getState( )
  - void setState(boolean *checked*)
- For add MenuItem:
  - MenuItem add(MenuItem *item*)
- For add MenuBar
  - Menu add(Menu *menu*)
- To get Item from Menu:
  - Object getItem( )

```
import java.awt.*;
class MenuExample
{
    MenuExample(){
      Frame f= new Frame("Menu Example");

            MenuBar mb=new MenuBar();
      Menu menu=new Menu("Menu");
      Menu submenu=new Menu("Sub Menu");

      MenuItem i1=new MenuItem("Item 1");
      MenuItem i2=new MenuItem("Item 2");
      MenuItem i3=new MenuItem("Item 3");
      MenuItem i4=new MenuItem("Item 4");
      MenuItem i5=new MenuItem("Item 5");
menu.add(i1);
menu.add(i2);  menu.add(i3);
submenu.add(i4);
submenu.add(i5);
menu.add(submenu);
mb.add(menu);
f.setMenuBar(mb);
f.setSize(400,400);
f.setLayout(null);
f.setVisible(true);
}
public static void main(String
 args[])  {
new MenuExample();
}  }
```


Menu and MenuItem Example
Menu
Item 1
Item 2
Item 3
Sub Menu
Item 4
Item 5

DialogBox

- Dialog boxes are primarily used to obtain user input.
- They are similar to frame windows, except that dialog boxes are always child windows of a top-level window.
- Dialog boxes don’t have menu bars.
- In other respects, dialog boxes function like frame windows.
- Dialog boxes may be modal or modeless.
- When a *modal* dialog box is active, all input is directed to it until it is closed.
- When a *modeless* dialog box is active, input focus can be directed to another window in your program.

- *Constructors:*.
  - Dialog(Frame *parentWindow*, boolean *mode*)
  - Dialog(Frame *parentWindow*, String *title*, boolean *mode*)
- To create Dialog Box:
  - Create Frame or Applet
  - Create another class which extends Dialog class.
  - Call this new class from Frame/Applet class.
  - In constructor of Extended Dialog class, use super method and pass vales to constructor of Dialog

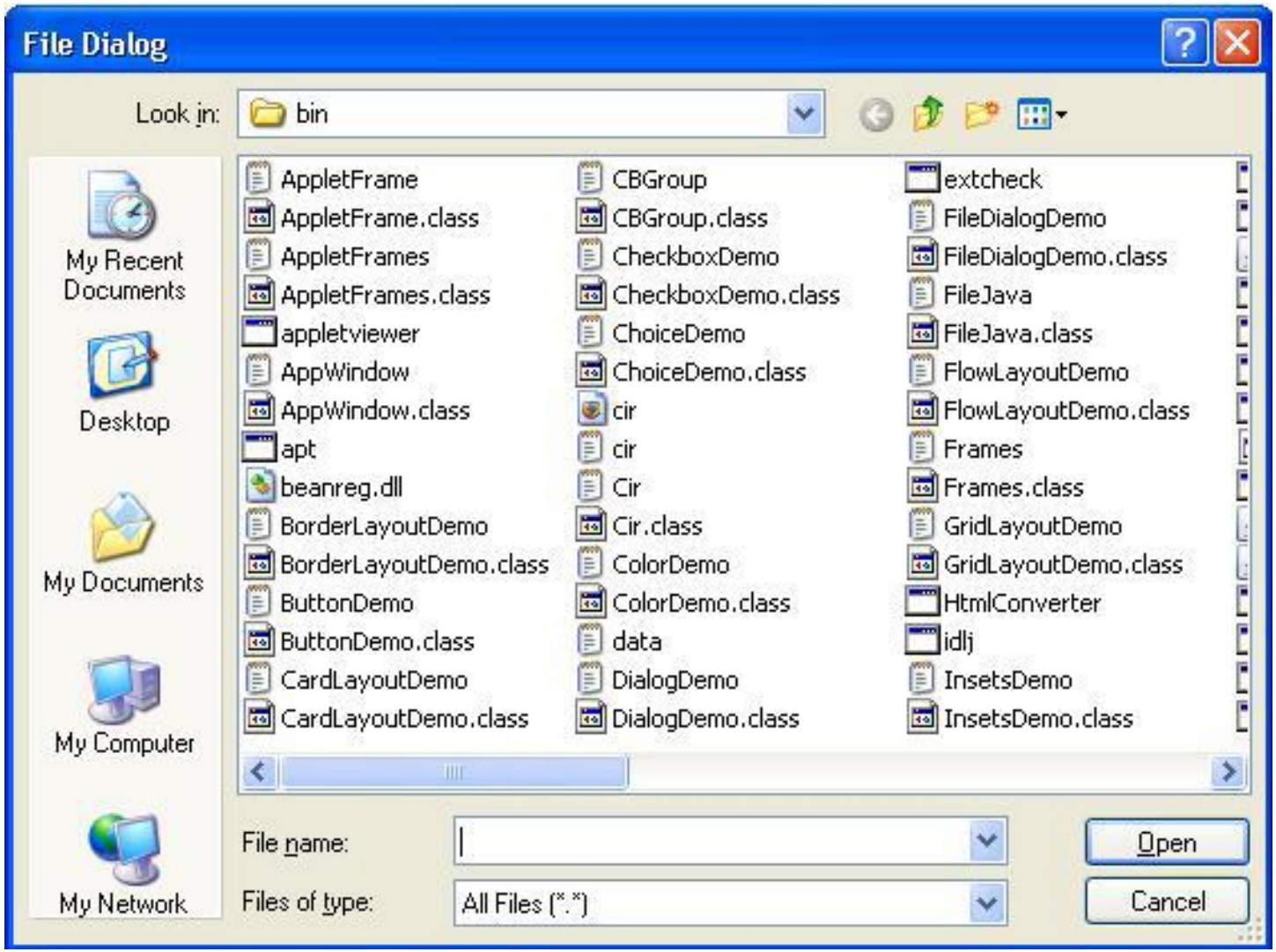
```
import java.awt.*;
import java.awt.event.*;
public class DialogExample {
   private static Dialog d;
   DialogExample() {
      Frame f= new Frame();
      d = new Dialog(f , "Dialog Example", true);
      d.setLayout( new FlowLayout() );
      Button b = new Button ("OK");
      b.addActionListener ( new ActionListener()
      {  public void actionPerformed( ActionEvent e )
         { DialogExample.d.setVisible(false); }
      });
      d.add( new Label ("Click button to continue."));
      d.add(b);          d.setSize(300,300);
      d.setVisible(true);     }
```



FileDialog

- Java provides a built-in dialog box that lets the user specify a file.
- To create a file dialog box, instantiate an object of type **FileDialog**.
- Constructor:
  - FileDialog(Frame *parent*, String *boxName*)
  - FileDialog(Frame *parent*, String *boxName*, int *how*)
  - FileDialog(Frame *parent*)
- Int how: **FileDialog.LOAD, FileDialog.SAVE**
- **Methods:**
  - String getDirectory( )
  - String getFile( )

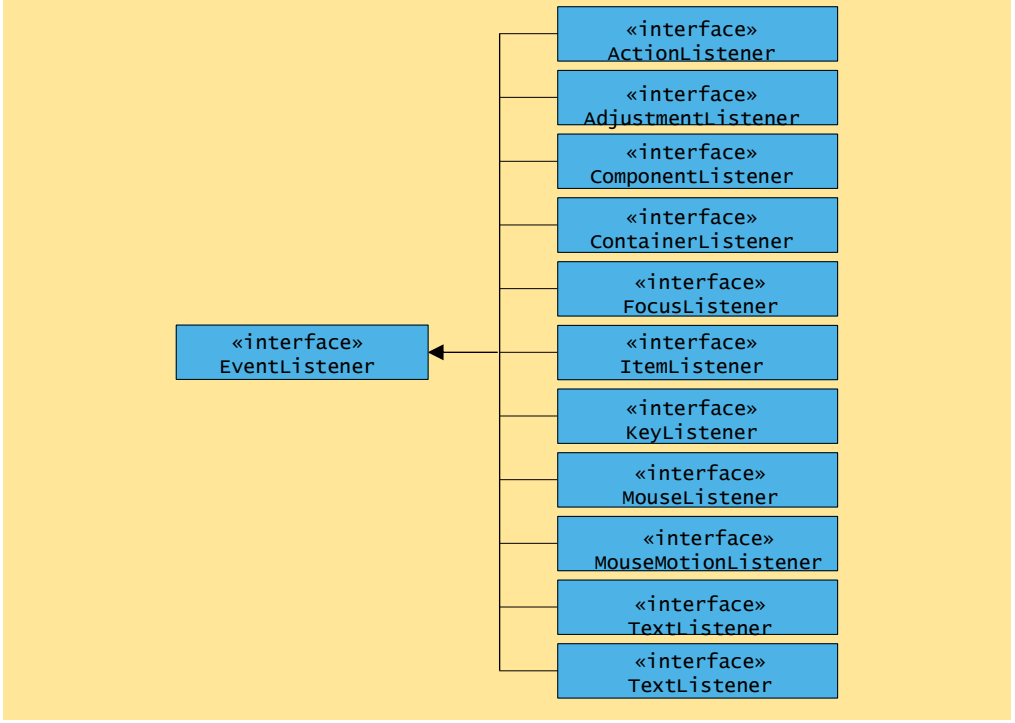
```
import java.awt.*;
class SampleFrame extends Frame
{
SampleFrame(String title){
super(title); }}
class FileDialogDemo
{public static void main(String args[]){
Frame f = new SampleFrame("File Dialog Demo");
f.setVisible(true);
f.setSize(100, 100);
FileDialog fd = new FileDialog(f, "File Dialog");
fd.setVisible(true);
}}
```


File Dialog
Look in:
bin
My Recent Documents
Desktop
My Documents
My Computer
My Network
AppletFrame
AppletFrame.class
AppletFrames
AppletFrames.class
appletviewer
AppWindow
AppWindow.class
apt
beanreg.dll
BorderLayoutDemo
BorderLayoutDemo.class
ButtonDemo
ButtonDemo.class
CardLayoutDemo
CardLayoutDemo.class
CBGroup
CBGroup.class
CheckboxDemo
CheckboxDemo.class
ChoiceDemo
ChoiceDemo.class
cir
cir
Cir
Cir.class
ColorDemo
ColorDemo.class
data
DialogDemo
DialogDemo.class
extcheck
FileDialogDemo
FileDialogDemo.class
FileJava
FileJava.class
FlowLayoutDemo
FlowLayoutDemo.class
Frames
Frames.class
GridLayoutDemo
GridLayoutDemo.class
HtmlConverter
idlj
InsetsDemo
InsetsDemo.class
File name:
Files of type:
All Files (*.*)
Open
Cancel

Event Listeners

- A listener is an object that is notified when an event occurs.
- Event has two major requirements.
    1. It must have been registered with one or more sources to receive notifications about specific types of events.
    2. It must implement methods to receive and process these notifications.
- The methods that receive and process events are defined in a set of interfaces found in java.awt.event.
- For example, the MouseMotionListener interface defines two methods to receive notifications when the mouse is dragged or moved.
- Any object may receive and process one or both of these events if it provides an implementation of this interface.

Event-listener interfaces of package `java.awt.event`



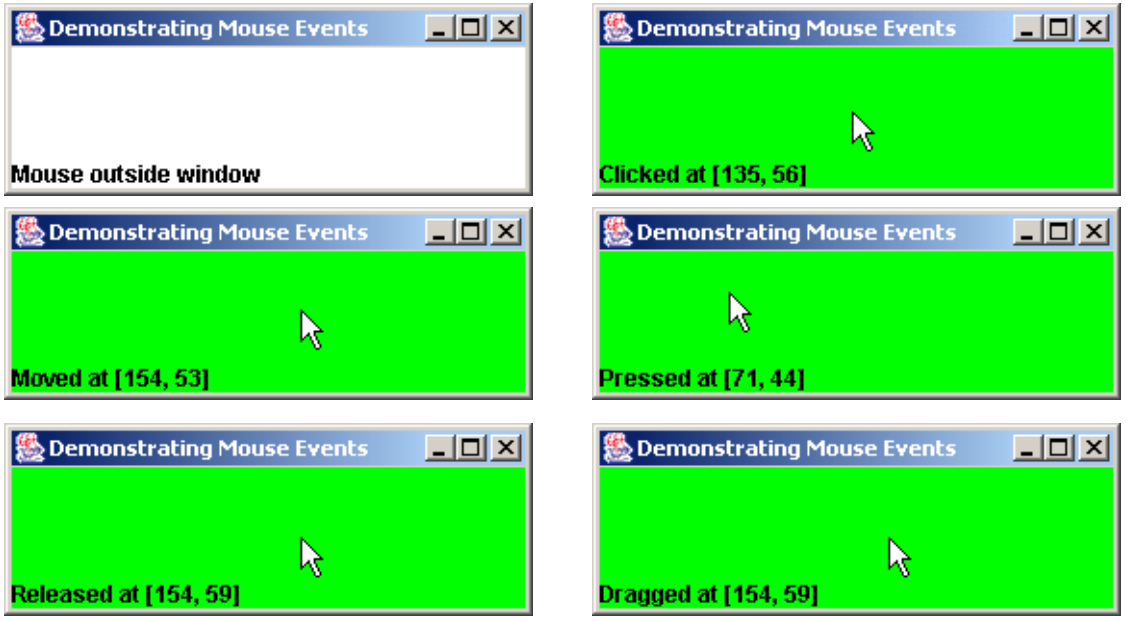
Handling mouse events

- Event-listener interfaces for mouse events
  - MouseListener
  - MouseMotionListener
  - Listen for MouseEvents

`MouseListener` and `MouseMotionListener` interface methods

`MouseListener` and **`MouseMotionListener`** interface methods	
Methods of interface `MouseListener`	
`public void mousePressed( MouseEvent event )`	
	Called when a mouse button is pressed while the mouse cursor is on a component.
`public void mouseClicked( MouseEvent event )`	
	Called when a mouse button is pressed and released while the mouse cursor remains stationary on a component.
`public void mouseReleased( MouseEvent event )`	
	Called when a mouse button is released after being pressed. This event is always preceded by a `mousePressed` event.
`public void mouseEntered( MouseEvent event )`	
	Called when the mouse cursor enters the bounds of a component.
`public void mouseExited( MouseEvent event )`	
	Called when the mouse cursor leaves the bounds of a component.
Methods of interface `MouseMotionListener`	
`public void mouseDragged( MouseEvent event )`	
	Called when the mouse button is pressed while the mouse cursor is on a component and the mouse is moved while the mouse button remains pressed. This event is always preceded by a call to `mousePressed`. All drag events are sent to the component on which the drag began.
`public void mouseMoved( MouseEvent event )`	
	Called when the mouse is moved when the mouse cursor on a component. All move events are sent to the component over which the mouse is currently positioned.

MouseListener & MouseMotionListener example



Handling keyboard events

- Keyboard events, can be handled by implementing the KeyListener interface.
- KeyListener interface defines three methods. The general forms of these methods are:
    1. void keyPressed(KeyEvent ke)
    2. void keyReleased(KeyEvent ke)
    3. void keyTyped(KeyEvent ke)
- To implement keyboard events implementation to the above methods is needed.

Adapter classes

- Java provides a special feature, called an adapter class, that can simplify the creation of event handlers.
- An adapter class provides an empty implementation of all methods in an event listener interface.
- Adapter classes are useful when you want to receive and process only some of the events that are handled by a particular event listener interface.
- You can define a new class to act as an event listener by extending one of the adapter classes and implementing only those events in which you are interested.

Adapter Classes

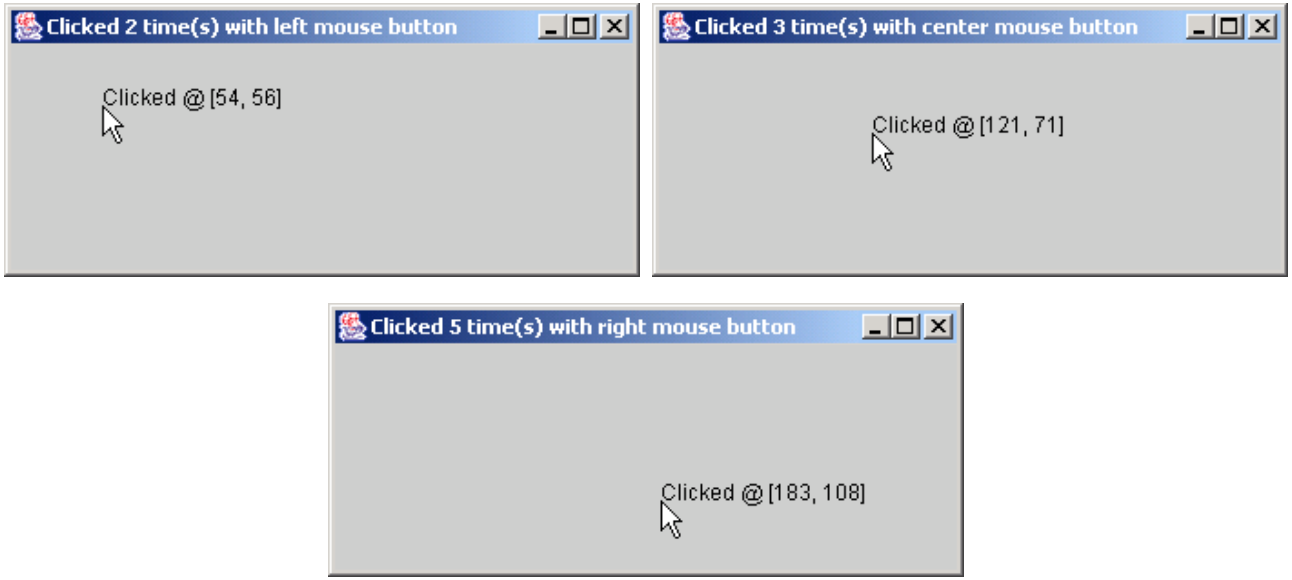
- Time consuming to define all interface methods
  - **`WindowListener`** has seven methods
    - What if we only want to use one?
    - Required to define all methods in interface
- Adapter class implements an interface
    - Does anyone recognize a design pattern here?
    - Default implementation ({ }, empty body) for all methods
  - You then extend adapter class,
    - overriding methods for events you care about, such as `windowClosing`.
  - Has "is a" relationship with interface
    - **`WindowAdapter`** is a **`WindowListener`**
    - **`MouseAdapter`** is a **`MouseListener`**

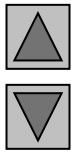
- Adapter classes in java.awt.event are

Event-adapter class	Implements interface
ComponentAdapter	ComponentListener
ContainerAdapter	ContainerListener
FocusAdapter	FocusListener
KeyAdapter	KeyListener
MouseAdapter	MouseListener
MouseMotionAdapter	MouseMotionListener
WindowAdapter	WindowListener

Adapter example

- The window detects and shows where it is clicked





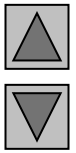
Outline

MouseDetails.java

Line 15

```
1  // Fig. 13.20: MouseDetails.java
2  // Demonstrating mouse clicks and distinguishing between mouse buttons.
3  import java.awt.*;
4  import java.awt.event.*;
5  import javax.swing.*;
6
7  public class MouseDetails extends JFrame {
8     private int xPos, yPos;
9
10    // set title bar String; register mouse listener; size and show window
11    public MouseDetails()
12    {
13       super( "Mouse clicks and buttons" );
14
15       addMouseListener( new MouseClickHandler() );
16
17       setSize( 350, 150 );
18       setVisible( true );
19    }
20
21    // draw String at location where mouse was clicked
22    public void paint( Graphics g )
23    {
24       // call superclass paint method
25       super.paint( g );
26
```

Register mouse listener



Outline

MouseDetails.java

Line 41

Lines 43-44

Line 46

Line 48

```
27        g.drawString( "Clicked @ [" + xPos + ", " + yPos + "]",
28           xPos, yPos );
29     }
30
31     public static void main( String args[] )
32     {
33        MouseDetails application = new MouseDetails();
34        application.setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE );
35     }
36
37     // inner class to handle mouse events
38     private class MouseClickHandler extends MouseAdapter {
39
40        // handle mouse click event and determine which button was pressed
41        public void mouseClicked( MouseEvent event )
42        {
43           xPos = event.getX();
44           yPos = event.getY();
45
46           String title = "Clicked " + event.getClickCount() + " time(s)";
47
48           if ( event.isMetaDown() ) // right mouse button
49              title += " with right mouse button";
50
51           else if ( event.isAltDown() ) // middle mouse button
52              title += " with center mouse button";
```

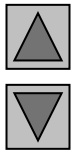
Invoke method `mouseClicked` when user clicks mouse

Store mouse-cursor coordinates where mouse was

Determine number of times user has clicked mouse

Determine if user clicked right mouse

Determine if user clicked middle mouse button



Outline

MouseDetails.java

```
53
54          else  // left mouse button
55             title += " with left mouse button";
56
57          setTitle( title );  // set title bar of window
58          repaint();
59
60       } // end method mouseClicked
61
62    } // end private inner class MouseClickHandler
63
64 } // end class MouseDetails
```

`InputEvent` methods that help distinguish among left-, center- and right-mouse-button clicks

`InputEvent` method	Description
`isMetaDown()`	Returns `true` when the user clicks the right mouse button on a mouse with two or three buttons. To simulate a right-mouse-button click on a one-button mouse, the user can hold down the *Meta* key on the keyboard and click the mouse button.
`isAltDown()`	Returns `true` when the user clicks the middle mouse button on a mouse with three buttons. To simulate a middle-mouse-button click on a one- or two-button mouse, the user can press the *Alt* key on the keyboard and click the only- or left-mouse button, respectively.

An Applet

- Applet is a public class which is predefined by **java.applet.Applet**
- There is no main() method in Applet like Application program. The main() method is defined by browser or Appletviewer for Applet.
- Life cycle methods: init, start, paint, stop, destroy
- Applet is one type of container and subclass of Panel.

Concepts of Applets

- Applets are small applications that are accessed on an Internet server, transported over the Internet, automatically installed, and run as part of a Web document.
- After an applet arrives on the client, it has limited access to resources, so that it can produce an arbitrary multimedia user interface and run complex computations without introducing the risk of viruses or breaching data integrity.
- applets – Java program that runs within a Java-enabled browser, invoked through an “applet” reference on a web page, dynamically downloaded to the client computer

```
import java.awt.*;
import java.applet.*;
public class SimpleApplet
extendsApplet {
public void paint(Graphics g) {
g.drawString("A SimpleApplet",
20, 20); } }
```

There are two ways to run an applet:

- Executing the applet within a Java-compatible Web browser, such as Netscape Navigator.
- Using an applet viewer, such as the standard JDK tool, applet viewer.
- An applet viewer executes your applet in a window. This is generally the fastest and easiest way to test an applet.
- To execute an applet in a Web browser, you need to write a short HTML text file that contains the appropriate APPLET tag.

Differences between applets and applications

- Java can be used to create two types of programs: applications and applets.
- An application is a program that runs on your computer, under the operating system of that Computer(i.e an application created by Java is more or less like one created using C orC++).
- When used to create applications, Java is not much different from any other computer language.
- An applet is an application designed to be transmitted over the Internet and executed by a Java-compatible Web browser.
- An applet is actually a tiny Java program, dynamically downloaded across the network, just like an image, sound file, or videoclip.

- The important difference is that an applet is an intelligent program, not just an animation or media file(i.e an applet is a program that can react to user input and dynamically change—not just run the same animation or sound over and over.
- Applications require main method to execute.
- Applets do not require main method.
- Java's console input is quite limited
- Applets are graphical and window-based.

Life cycle of an applet

Applets life cycle includes the following methods

- init()
- start( )
- paint( )
- stop( )
- destroy( )

When an applet begins, the AWT calls the following methods, in this sequence: init( ) start( ) paint()
When an applet is terminated, the following sequence of method calls takes place: stop( ) destroy( )

- init( ): The init( ) method is the first method to be called. This is where you should initialize variables. This method is called only once during the run time of your applet.
- start( ): The start( ) method is called after init( ). It is also called to restart an applet after it has been stopped. Whereas init( ) is called once—the first time an applet is loaded—start( ) is called each time an applet's HTML document is displayed onscreen. So, if a user leaves a web page and comes back, the applet resumes execution at start( ).
- paint( ): The paint( ) method is called each time applet's output must be redrawn. paint( ) is also called when the applet begins execution. Whatever the cause, whenever the applet must redraw its output, paint( ) is called. The paint( ) method has one parameter of type Graphics. This parameter will contain the graphics context, which describes the graphics environment in which the applet is running. This context is used whenever output to the applet is required.
- stop( ): The stop( ) method is called when a web browser leaves the HTML document containing the applet—when it goes to another page, for example. When stop( ) is called, the applet is probably running. Applet uses stop( ) to suspend threads that don't need to run when the applet is not visible. To restart start( ) is called if the user returns to the page.
- destroy( ): The destroy( ) method is called when the environment determines that your applet needs to be removed completely from memory. The stop( ) method is always called before destroy( ).

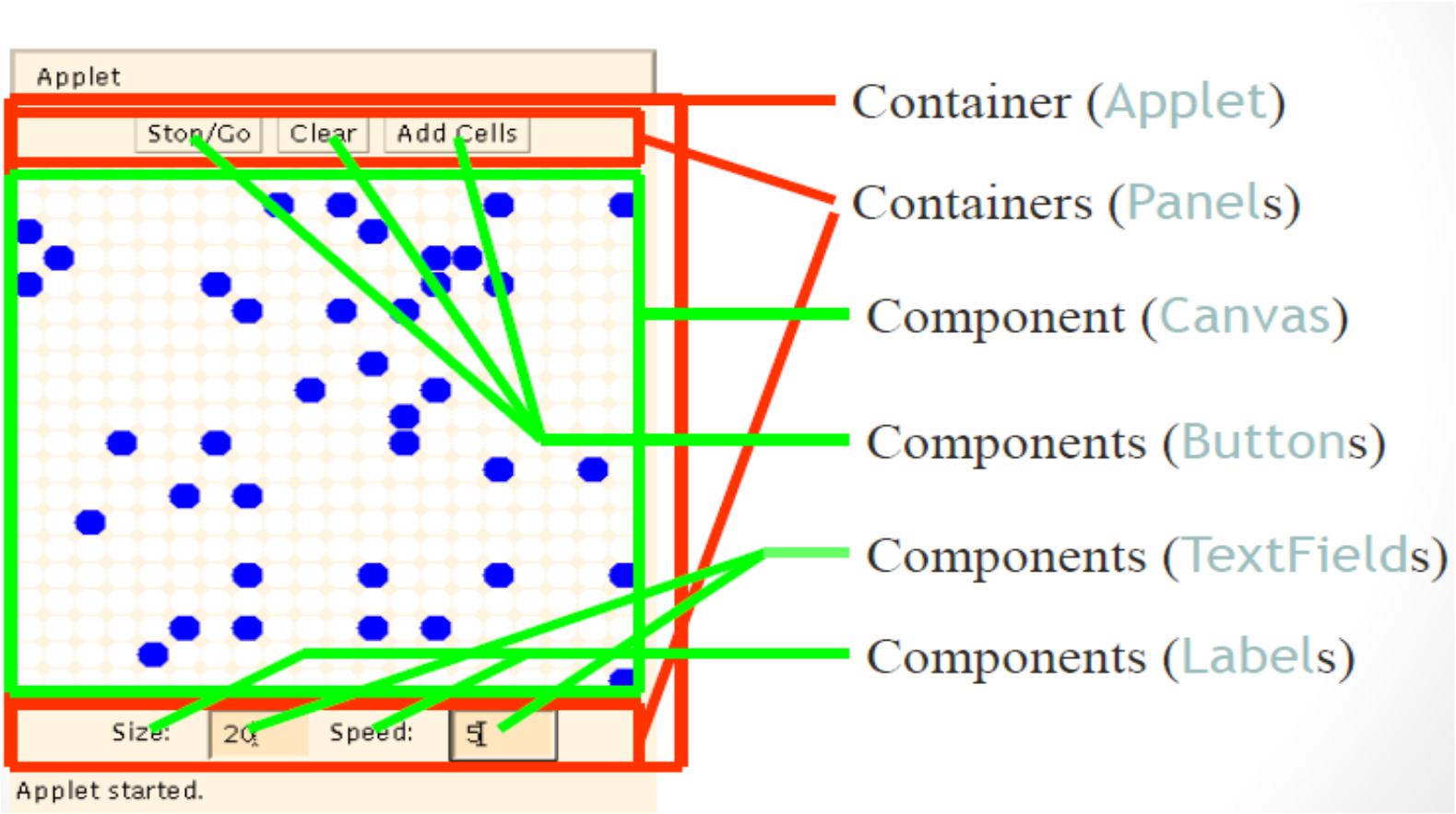
Types of applets

- Applets are two types
  - Simpleapplets
  - JApplets
- Simple applets can be created by extending Applet class
- JApplets can be created by extending JApplet class of javax.swing.JApplet package

To create an applet

- import java.applet.*;
- Import java.awt.*;
- Applet tag code in comment.
- Extends Applet class
- Life cycle method
- Class must be public

Applet Life Cycle



Creating applets

Applets are created by extending the Applet class.

```
import java.awt.*;
import java.applet.*; /* */
public classAppletSkel extends Applet{
    publicvoid init() { // initialization }
    publicvoid start() { // startorresume execution }
    publicvoid stop() { // suspends execution }
    publicvoid destroy() { // perform shutdownactivities }
    publicvoid paint(Graphics g) { // redisplaycontents of window
    }
    }
```

passing parameters to applets

- APPLET tag in HTML allows you to pass parameters to applet.
- To retrieve a parameter, use the getParameter( ) method. It returns the value of the specified parameter in the form of a String object. // Use Parameters

```
import java.awt.*;
importjava.applet.*; /* */
```

- public class ParamDemoextends Applet{

```
String fontName;
int fontSize;
Float leading;
boolean active; // Initialize the string to be displayed.
public void start() {
        String param;
        fontName = getParameter("fontName");
        if(fontName== null)
                fontName= "Not Found";
        param = getParameter("fontSize");
        try {
                if(param != null) // if not found
                fontSize =Integer.parseInt(param);
                else
                        fontSize = 0;
        } catch(NumberFormatException e) { fontSize = -1; }
        param = getParameter("leading");
```

```
try { if(param != null) // if notfound
        leading = Float.valueOf(param).floatValue();
        else leading = 0;
} catch(NumberFormatException e) { leading = -1; }
param = getParameter("accountEnabled");
if(param != null)
active = Boolean.valueOf(param).booleanValue(); }
// Displayparameters.
public void paint(Graphics g) {
        g.drawString("Font name: " + fontName, 0, 10);
        g.drawString("Font size: " + fontSize, 0, 26);
        g.drawString("Leading: " + leading, 0, 42);
        g.drawString("AccountActive: " + active, 0, 58);
}
}
```

Introduction to swings

- Swing is a set of classes that provides more powerful and flexible components than are possible with the AWT.
- In addition to the familiar components, such as buttons, check boxes, and labels, Swing supplies several exciting additions, including tabbed panes, scroll panes, trees, and tables.
- Even familiar components such as buttons have more capabilities in Swing.
- For example, a button may have both an image and a text string associated with it. Also, the image can be changed as the state of the button changes.
- Unlike AWT components, Swing components are not implemented by platform-specific code.
- Instead, they are written entirely in Java and, therefore, are platform independent.
- The term lightweight is used to describe such elements.

The Swing component are defined in javax.swing

1. AbstractButton: Abstract superclass for Swing buttons.
2. ButtonGroup: Encapsulates a mutually exclusive set of buttons.
3. ImageIcon: Encapsulates an icon.
4. JApplet: The Swing versionof Applet.
5. JButton: The Swing push button class.
6. JCheckBox: The Swing check box class.
7. JComboBox : Encapsulates a combo box (an combination of a drop-down list and text field).
8. JLabel: The Swing versionof a label.
9. JRadioButton: The Swing versionof a radio button.
10. JScrollPane: Encapsulates a scrollable window.
11. JTabbedPane: Encapsulates a tabbed window.
12. JTable: Encapsulates a table-based control.
13. JTextField: The Swing versionof a text field.
14. JTree: Encapsulates a tree-based control.

Limitations of AWT

- AWT supports limited number of GUI components.
- AWT components are heavyweight components.
- AWT components are developed by using platform specific code.
- AWT components behaves differently in different operating systems.
- AWT component is converted by the native code of the operating system.

- Lowest Common Denominator
  - If not available natively on one Java platform, not available on any Java platform
- Simple Component Set
- Components Peer-Based
  - Platform controls component appearance
  - Inconsistencies in implementations
  - Interfacing to native platform error-prone

Event Handling Summary

Interface	**Methods**	**Parameter**	**Events Generated by**
ActionListener	actionPerformed	ActionEvent getActionCommand getmodifiers	Button List MenuItem TextField
Adjustment Listner	adjustmentValue Changed	AdjustmentEvent getAdjustable getAdjustmentType getvalue	Scrollbar
ItemListner	itemStateChanged	ItemEvent getItem getItemSelectable getstateChange	Checkbox CheckboxMenu Item Choice List
TextListner	textValue Changed	TextEvent	TextComponent

Event Handling Summary

Interface	Methods	Parameter	Events Generated by
Component Listener	componentMoved componentHidden componentResized	ComponentEvent getComponent	Component
Container Listener	componentAdded componentRemoved	ContainerEvent getChild getContainer	Container
FocusListener	focusGained focusLost	focusEvent IsTemporary	Component
KeyListener	keyPressed keyRealsed keyTyped	KeyEvent getKeyChar getKeyCode getkeyModifiersText isActionKey	Component

Event Handling Summary

Interface	Methods	Parameter	Events Generated by
Mouse Listener	mousePressed mouseRealesed mouseEntered mouseExited mouseClicked	MouseEvent getClickCount getX getY getPoint translatePoint isPopupTrigger	Component
MouseMotion Listener	mouseDragged mouseMoved		Component
Window Listener	windowClosing windowOpened windowIconed windowDeiconed windowClosed windowActivated windowDeactivated	WindowEvent getWindow	Window